ECE 408

第 15 讲

Parallel Computation Patterns: Reduction Trees

目录

1 归约的运算结构与并行代价 **1**
1.1 二元运算、单位元与结果的含义 1
1.2 从顺序依赖链到归约树 2
1.3 原子累加与局部合并 3

2 分块归约与内存访问 **4**
2.1 数据分段与同步边界 4
2.2 递增与递减步长的线程映射 4
2.3 在共享内存中保存中间结果 7

3 线程束内通信与两阶段归约 **9**
3.1 线程束内的寄存器交换 9
3.2 用线程束交换完成归约树的末段 10
3.3 显式线程组与协作组 11
3.4 先在各线程束内归约，再合并线程束结果 13

4 线程粗化与完整归约实现 **15**
4.1 增加每个线程承担的局部工作 15
4.2 工作量、阶段数与同步次数的比较 16
4.3 完整核函数与块间合并 17

5 综合算例与关键结论 **20**
5.1 非整块输入的完整归约过程 20
5.2 大规模输入需要多少轮核函数 21
5.3 归约结构与开销的联系 22

1 归约的运算结构与并行代价

1.1 二元运算、单位元与结果的含义

归约（reduction）把一组输入通过同一种二元运算合并成一个结果。设输入包含 N 个值，依次为 $x_0, x_1, \dots, x_{N-1}$，二元运算记为 $\circ$，则结果为 $r = x_0 \circ x_1 \circ \cdots \circ x_{N-1}$。求和、求积、最小值和最大值都具有这种结构。与归约相关的前缀扫描（prefix scan）会输出各个前缀的累计结果，例如求和扫描在位置 k 输出 $\sum_{i=0}^{k} x_i$。两者共享规则化的合并结构，归约只保留最终的一个值。

并行化需要明确运算允许怎样重排。**结合律** $(a \circ b) \circ c = a \circ (b \circ c)$ 允许改变括号结构，从而把一条依赖链改写成一棵树；**交换律** $a \circ b = b \circ a$ 允许交换元素顺序。后面按两半配对、再按线程束合并的求和实现都会改变分组，某些映射还会改变合并次序，因此采用满足这两条性质的理想运算模型。一般而言，保持原有顺序的归约树只依赖结合律；要允许任意顺序重排，还需要交换律。

单位元 e 满足 $e \circ a = a \circ e = a$。它既用于初始化累计量，也用于把不足一个完整数据段的位置补齐。常用运算与单位元如表1 所示。对于浮点最小值和最大值，这里只讨论普通有序数值，特殊非数值的处理需要另行指定。

表 1 常见归约运算及其单位元

运算	二元合并	单位元	补齐位置的含义
求和	$a + b$	0	不增加总和
求积	ab	1	不改变已有乘积
最小值	$\min(a, b)$	$+\infty$ 或类型上界	不成为更小的候选
最大值	$\max(a, b)$	$-\infty$ 或类型下界	不成为更大的候选

顺序归约可以先令 $a = e$，再对每个输入更新 $a \leftarrow a \circ x_i$。这需要 N 次更新；若 $N \geq 1$，也可以直接用 x_0 初始化，只合并余下的 $N - 1$ 个值。两种写法的时间复杂度均为 $\Theta(N)$，后者便于与树上必需的 $N - 1$ 次二元合并比较。空输入则按单位元约定返回 e。

整数求和也需要明确数值范围。数学整数的结合律不意味着固定宽度类型可以忽略溢出：有符号整数的某个中间部分和溢出时，C++ 运算不再具有预期语义，即使最终数学结果可表示也不足以保证安全。使用无符号类型时，加法按位宽取模；只有所有累计值均在计数范围内，模运算结果才等于通常的整数总和。

对实数加法成立的结合律，在有限精度浮点运算中通常只能近似成立。以单精度数值为例，$(10^{20} + (-10^{20})) + 1$ 可以得到 1，而 $10^{20} + ((-10^{20}) + 1)$ 可以得到 0，因为小量 1 在与

大幅值负数相加时被舍入掉。归约树改变括号结构，块间原子累加还可能改变执行次序，因此正确的并行求和也可能与顺序求和产生数值差异；存在严重消去时，这种差异可能明显增大。验证时应依据数据尺度比较绝对误差和相对误差；若任务要求固定的浮点结果，则还需要固定跨块合并次序及运算实现。

1.2 从顺序依赖链到归约树

以下并行代价讨论取 $N \geq 2$。对于 $N = 2^L$ 个输入，平衡二叉归约树在第一级完成 $N/2$ 次两两合并，第二级完成 $N/4$ 次合并，直到最后一级只合并两个部分结果。同一级的各个操作互不依赖，后一层只等待自己的两个子结果。图1 用八个数的求和说明这一过程。

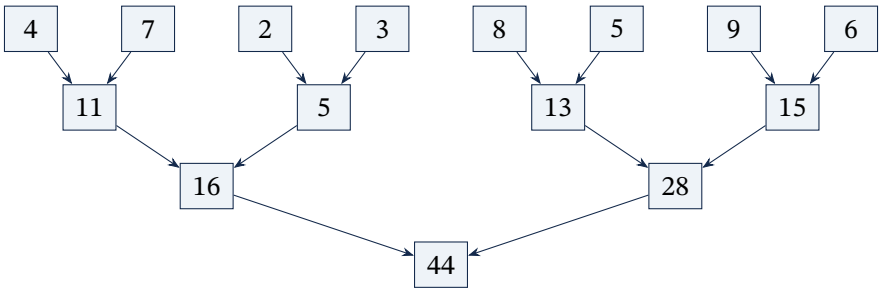


图 1 八个输入通过三级合并得到总和。每层分别执行四次、两次和一次加法。

用**工作量**（work）$W(N)$ 表示所有线程执行的必要二元合并总数，用**跨度**（span）$D(N)$ 表示最长依赖路径上的合并数。在单位合并时间的抽象模型中，归约树满足

$$W(N) = \sum_{k=1}^{L} \frac{N}{2^k} = N - 1, \qquad D(N) = L = \log_2 N. \tag{1}$$

工作量与优化过的顺序算法相同，因此这棵树是工作高效的（work-efficient）。这里比较的是必要合并数；索引计算、数据搬移、同步和后面线程束代码中的冗余通道运算仍然具有实际开销。

若 N 不是二的幂，可以补单位元到 $\widehat{N} = 2^{\lceil \log_2 N \rceil}$。规则树的深度为 $\lceil \log_2 N \rceil$，工作量为 $\widehat{N} - 1 < 2N - 1$；若只合并实际存在的值，仍只需要 $N - 1$ 次合并。后面的分块实现通过边界判断补零，无需真的建立一份全局填充数组。

平均可用并行度为 W/D。当 N 是二的幂时，它等于 $(N-1)/\log_2 N$，峰值出现在第一级，为 $N/2$。例如，对 $N = 1{,}000{,}000$ 个实际输入建立平衡归约树时，深度为 $\lceil \log_2 N \rceil = 20$，工作量为 $N - 1$，平均并行度约为 50,000，第一级包含 500,000 次独立合并。这些数量说明，要把树深度直接兑现成运行时间，需要非常多的执行资源。

若硬件同一时刻最多执行 P 次合并，则 $T_P \geq \max\{W/P, D\}$；忽略通信等开销的理想调度时间量级为 $\Theta(W/P + D)$。因此，$\Theta(\log N)$ 描述依赖深度，并不能单独给出实际图形处理器（GPU）的运行时间。树越接近根，活跃操作越少，后期必然出现并行度收缩。一个核函数启动后线程块的线程数固定，停止做加法的线程也不会自动变成新线程去处理下一批数据。

1.3 原子累加与局部合并

最直接的并行求和方式，是让每个线程读取一个输入，再用原子操作（atomic operation）把它加到同一个全局输出上。原子加法把一次读、加和写作为不可分割的更新，保证同时到达的线程不会因覆盖彼此的写入而丢失贡献。

代码 1 逐元素原子求和：同一个输出地址接收全部更新

```
__global__ void reduce_atomic(const float* input,
                              size_t n, float* sum) {
    size_t i = size_t(blockIdx.x) * blockDim.x
             + threadIdx.x;
    if (i < n) {
        atomicAdd(sum, input[i]);
    }
}
```

这段核函数要求输出地址有效，并且在核函数执行前将 `*sum` 设为 0。输出不能与仍待读取的输入区域重叠。即使所有输入读取都能并行，同一地址上的 N 次原子更新仍然相互竞争；这些更新必须以某种顺序生效，难以获得归约树同一级那样的独立合并能力。原子操作保证更新正确，并不会自动消除这个共享地址形成的瓶颈。

一种直接的改进是先让每个线程块完成局部归约，只由该块的一个线程原子累加块和。若每块处理 $2B$ 个元素，块数为 $G = \lceil N/(2B) \rceil$，则全局原子更新数从 N 降为 G。例如 $N = 1{,}048{,}576$、$B = 256$ 时，只需 2048 次块结果更新。局部树负责把大量独立加法分散到各块中，最终的原子操作只连接这些块的结果。

另一种块间合并方法是先把 G 个块和写入独立数组，再启动后续归约核函数，直到只剩一个值。同一执行流中的后续核函数会在先前核函数完成后使用这些部分和。这两种方案共享相同的块内算法，区别集中在最终结果如何跨块汇合；后面将分别给出它们的使用条件。

2 分块归约与内存访问

2.1 数据分段与同步边界

归约树相邻两层之间存在数据依赖：后一层要读取前一层写出的部分和。线程块内的屏障 `__syncthreads()` 使块内参与线程在该点会合，并使屏障前的共享内存或全局内存访问对块内后续访问正确可见。普通核函数中的这个屏障只作用于本块，不能等待其他线程块完成它们的树层。

因此，大输入通常按块分段。统一记总输入长度为 N，每块线程数为 B，块号为 b，块内线程号为 t。未粗化时每块处理 $2B$ 个元素，段首为 $s_b = 2Bb$，块数为 $G = \lceil N/(2B) \rceil$。各块在自己的数据段内部完成归约，再把块和交给原子累加或后续核函数。这里将一次全局归约拆成块级局部归约，各段最终仍合成一个结果。通常所说的分段归约（segmented reduction）则为每个给定分段分别输出一个结果，输出含义与本讲任务不同。

局部树的所有线程都必须遵守同一层间同步结构。即使某个线程已经不再执行加法，只要它仍在这一固定块内算法中，就继续经过对应的块屏障。因此，`__syncthreads()` 放在活动线程的条件分支之后。把屏障移入 `if (t < stride)` 内部，会使仍在执行的同块线程走向不一致的同步位置。

下面先用完整数据段比较两种线程映射。两段原地代码均要求 B 为二的幂、启动时恰好使用 B 个线程，且输入缓冲区具有完整的 $2BG$ 个元素；若 $N < 2BG$，应预先在缓冲区尾部补 0。它们会覆盖输入，并使用单独的块结果数组。之后的共享内存实现将显式检查 N，从而直接处理未补齐的尾块。

2.2 递增与递减步长的线程映射

第一种映射让线程 t 从相邻的两个位置 $s_b + 2t$ 和 $s_b + 2t + 1$ 开始。第一次合并后，部分和存放在偶数下标；下一次再隔开两个位置合并。步长 s 按 $1, 2, 4, \ldots, B$ 递增，参与条件为 $t \bmod s = 0$。固定的写入位置 $i = s_b + 2t$ 与逐渐增大的第二操作数位置 $i + s$ 共同形成相邻子树的合并。

代码 2 递增步长：在全局输入数组中原地合并

```
template<int B>
__global__ void reduce_interleaved(float* data,
                                   float* partial) {
    unsigned int t = threadIdx.x;
    size_t base = size_t(2) * B * blockIdx.x;
    size_t i = base + 2 * t;
    for (unsigned int s = 1; s <= B; s *= 2) {
        if (t % s == 0) {
            data[i] += data[i + s];
        }
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = data[base];
}
```

当步长为 s 的一层结束时，每个保留下来的位置保存连续 $2s$ 个原始元素之和。下一层把两个相邻的长度 $2s$ 的区间合并成长度 $4s$ 的区间，因此可以逐层推出最终段首保存全部 $2B$ 个元素之和。图2 给出 $B = 8$ 时的下标变化。

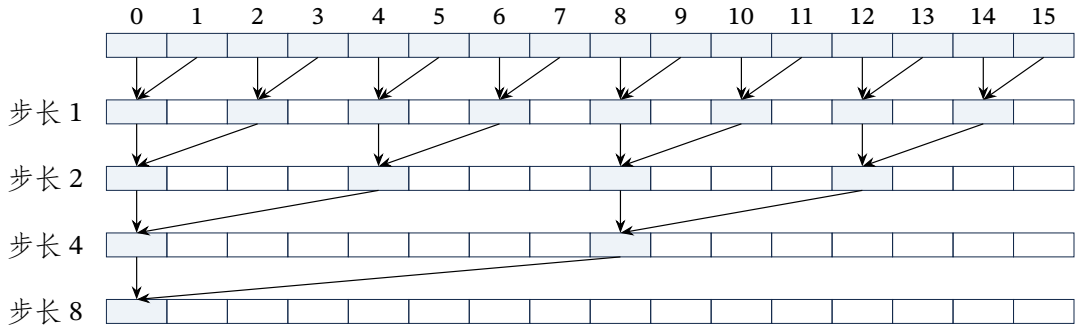


图 2 递增步长归约。蓝色位置保存仍会被后续层使用的部分和，存放位置逐层变得稀疏。

这套映射有两个相互独立的性能问题。首先，合并访问（coalesced access）关注*同一条内存指令下，相邻线程访问的地址*。在第一层，线程 $0, 1, 2, \ldots$ 的第一操作数分别位于 $0, 2, 4, \ldots$，第二操作数分别位于 $1, 3, 5, \ldots$。每个线程读取的两个位置虽然相邻，每条读指令在整个线程束上的地址却带有空隙，通常需要更多内存事务才能服务相同数量的有效数据。

其次，一个线程束（warp）由连续的 32 个线程组成。步长增大后，活动线程变为 $0, 2, 4, \ldots$，再变为 $0, 4, 8, \ldots$，使许多线程束内同时存在执行加法和跳过加法的线程。这种控制分歧（control divergence）降低了每条指令的有效通道利用率。直到较晚阶段，部分线程束才会整体停止算术工作。

第二种映射让线程 t 首先合并 $s_b + t$ 与 $s_b + t + B$，即将整个数据段的*左右两半对应相加*。随后步长按 $B, B/2, \ldots, 1$ 递减，活动线程始终是连续前缀 $0 \le t < s$。

代码 3 递减步长：使活动线程和保存位置保持连续

```
template<int B>
__global__ void reduce_contiguous(float* data,
                                  float* partial) {
    unsigned int t = threadIdx.x;
    size_t base = size_t(2) * B * blockIdx.x;
    size_t i = base + t;
    for (unsigned int s = B; s > 0; s /= 2) {
        if (t < s) {
            data[i] += data[i + s];
        }
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = data[base];
}
```

每一层把长度 $2s$ 的有效前缀缩短为长度 s 的前缀。两半是互不重叠的部分和集合，合并后仍包含相同的一组原始输入，因而最终总和保持不变。完整的 $2B$ 元素归约一共有 $1+\log_2 B$ 层；只有在首次 $2B \to B$ 的合并已经完成之后，剩余层数才是 $\log_2 B$。

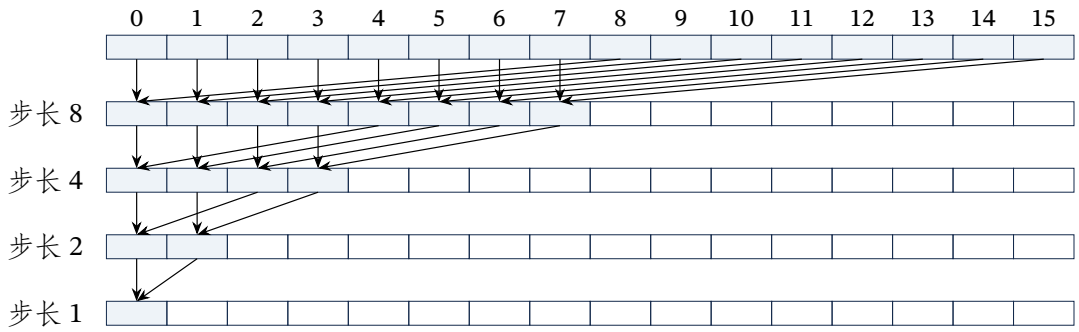


图 3 递减步长归约。每层把右半部分并入左半部分，有效部分和始终紧凑地位于前缀中。

在相同的读取指令下，活动线程的地址现在依次是 $s_b + t$ 或 $s_b + t + s$，两组访问各自连续。只要起始地址满足相应对齐条件，这种映射有利于合并访存。在活动线程数至少为 32 且为 32 的倍数时，活动范围包含若干完整线程束，因此每个线程束内部的分支结果一致；到最后不足 32 个线程工作时，分歧才集中出现在第一个线程束中。

例如 $B = 128$ 时，两种算法各层的活动线程数都依次为 $128, 64, 32, 16, 8, 4, 2, 1$，但含有活动线程的线程束数不同。表2 只统计执行加法的线程束，未参与加法的线程仍会按代码经过块屏障。

还可以把表中的活动线程束数换算成算术通道利用率。只考虑这些加法指令，将每个含有活动线程的线程束按 32 个通道计费，则递增步长占用的通道槽位为 $32(4+4+4+4+4+4+2+1) = 864$，递减步长为 $32(4+2+1+1+1+1+1+1) = 384$。两者均完成 $256-1 = 255$ 次必要加法，因此利用率分别为 $255/864 \approx 29.5\%$ 和 $255/384 \approx 66.4\%$。这项指标量化了线程

表 2 每块 128 个线程时，两种映射的算术活动分布

树层序号	0	1	2	3	4	5	6	7
活动线程数	128	64	32	16	8	4	2	1
递增步长的活动线程束数	4	4	4	4	4	4	2	1
递减步长的活动线程束数	4	2	1	1	1	1	1	1

映射造成的有效通道比例，不计访存、屏障和调度，也不是 SM 占用率或实测加速比。

两种树具有相同的必要加法次数和依赖深度。性能差别来自这些操作如何分配给线程、每次访存服务多少有效数据，以及线程束内多少通道实际做了有用工作。因此，单看 $O(N)$ 工作量与 $O(\log N)$ 深度，还无法区分这两段代码的性能。

2.3 在共享内存中保存中间结果

归约过程会不断读写存放部分和的位置。虽然数值每层都在变化，这些中间存储位置会被反复使用，因此适合放入由同一线程块访问的共享内存（shared memory）。让全局内存只提供原始输入并接收块结果，就可以减少归约树内部对全局地址的读写，也能保留原输入以供后续计算使用。

首次加载可以直接完成 $2B \to B$ 的合并：线程 t 读取两个原始值，在寄存器中相加，再把结果写入共享数组的第 t 项。这样共享数组只需要 B 项，而无需保存 $2B$ 个原始输入。所有槽位写入并同步后，再对这 B 项运行递减步长归约。

代码 4 带边界检查的共享内存归约：每块输出一个部分和

```
template<int B>
__global__ void reduce_shared(const float* input,
                              size_t n, float* partial) {
    static_assert(B > 0 && (B & (B - 1)) == 0,
                  "B must be a power of two");
    __shared__ float values[B];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * B * blockIdx.x + t;

    float a = (i < n) ? input[i] : 0.0f;
    float b = (i + B < n) ? input[i + B] : 0.0f;
    values[t] = a + b;
    __syncthreads();

    for (unsigned int s = B / 2; s > 0; s /= 2) {
        if (t < s) values[t] += values[t + s];
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = values[0];
}
```

该核函数使用一维网格和一维线程块，并要求实际启动线程数等于模板参数 B，同时符合设备的块大小限制。输出数组至少容纳 G 项，且与输入区间不重叠；例如另一块尚未完成读取时，不能把紧凑块结果写回输入数组前部。共享数组中每个槽位都由对应线程初始化；两个输入地址独立进行边界判断，保证尾块中的有效输入不会被遗漏。

以 $B = 128$、某一尾块仅剩 150 个有效输入为例，线程 0 至 21 各读取两个有效值，线程 22 至 127 各读取一个有效值并配上一个 0。整个块仍初始化 128 个共享槽位，并沿同一棵树归约。线程是否读到真实输入，与线程是否参与后续集体计算是两件事；补零使缺少输入的槽位仍具有定义良好的值，也保持了固定的同步结构。

初始化后的第一道屏障保证任何线程读取别人的共享槽位时，该槽位已经写好。循环中，每层的写入目标是前半个有效区间，第二操作数来自后半个有效区间；不同活跃线程的写入位置互不相同，且不会覆盖同层其他线程所需的第二操作数。层末屏障又保证后一层看到已经完成的前一层结果。这个不变量同时给出正确性依据和同步位置的依据。

对一个完整块，按源码中的标量操作计数，原地全局内存树有 $2B - 1$ 次必要加法，每次涉及两次读和一次写；共享内存版本只从全局输入读取 $2B$ 个值，再向块结果数组写一个值。缓存、寄存器保留和内存事务合并会影响实际访存次数，因此这种源码级比较用于识别访问结构，不能直接当作显存总线上的字节测量。

若输入为四字节单精度数，全部 N 个输入各读取一次、G 个块结果各写入一次，对应 $4N + 4G$ 字节的有效全局数据。所做加法与 N 同阶，所以数据搬移、同步以及指令利用率容

易主导耗时。共享内存优化解决了中间结果的存放问题，树后半段的块级同步和活动通道减少仍然存在。

在上面的直接写法中，每块执行一次初始化屏障和 $\log_2 B$ 次循环屏障，共 $1+\log_2 B$ 次。最后一层之后的屏障对只由线程 0 读取自身结果而言可以省略，这里保留统一的循环形式，便于后续比较。进一步减少同步，需要利用线程束内部更小的通信范围。

3 线程束内通信与两阶段归约

3.1 线程束内的寄存器交换

共享内存归约到后几层时，真正执行加法的线程已经集中到第一个线程束。此时，所需通信范围只有同一线程束内的线程，可以使用线程束交换（warp shuffle）直接传递寄存器中的数值。与先写共享内存、同步、再读取相比，这种方式减少了中间存储访问，也使协调范围缩小到参与交换的线程束。

记线程在束内的通道编号（lane ID）为 $\ell = t \bmod 32$，其寄存器值为 v_ℓ。交换指令让线程获得另一个通道在相应调用中提供的值，而不用把数值先写到共享数组。表3 按宽度为 32 的一个线程束给出四种常用寻址方式，d 为偏移或异或掩码，k 为指定的源通道，取 $0 \le k < 32$。

表 3 线程束交换原语的寻址关系

原语	源通道	作用
`__shfl_sync`	k	从指定通道取值，可用于广播
`__shfl_up_sync`	$\ell - d$	从编号更小的通道取值
`__shfl_down_sync`	$\ell + d$	从编号更大的通道取值，适合向第 0 通道汇合
`__shfl_xor_sync`	$\ell \oplus d$	按编号的按位异或关系交换

在向上或向下交换时，源编号越过当前宽度的边界会返回调用线程自身提供的值，编号不会环绕。交换宽度可以设为不超过 32 的二的幂，用于把线程束划分为连续的小组；后面的正式代码统一采用默认宽度 32。

交换调用的参与掩码（mask）描述当前线程束中哪些通道参与协调。`0xffffffffu` 的 32 位全为 1，对应完整线程束。下面统一要求一个线程束内 32 个线程都执行相同位置的交换调用，并使用相同掩码；被读取的源通道也必须实际参与相应调用。掩码不会把不存在的输入变成 0，也不会把稀疏的通道重新排成连续编号。因此，尾部输入的补零在交换之前完成。

代码 5 完整线程束内求和：仅第 0 通道的返回值用于输出

```
__device__ __forceinline__ float warp_sum(float value) {
    for (int offset = 16; offset > 0; offset /= 2) {
        value += __shfl_down_sync(0xffffffffu,
                                  value, offset);
    }
    return value;
}
```

这个函数依次使用偏移 $16, 8, 4, 2, 1$，共五轮交换和加法。第一次使低编号的前 16 个通道各合并两个值，第二次使前 8 个通道各覆盖四个原始值，依此类推，最终第 0 通道覆盖全部 32 个输入。所有通道都执行代码，函数只保证第 0 通道的返回值是整束总和；其他通道的值不能统一当作完整和或一般后缀和使用。

用一个完整的八通道子组可以手算同样的递减偏移规则。令初值依次为 $4, 7, 2, 3, 8, 5, 9, 6$，子组宽度设为 8，偏移依次为 $4, 2, 1$。表4 只列出仍会沿归约树进入第 0 通道最终结果的部分和，所有子组成员仍参与每轮交换。

表 4 八通道子组的交换归约手算

阶段	与最终结果相关的部分和	第 0 通道的值
初始化	$4, 7, 2, 3, 8, 5, 9, 6$	4
偏移 4	$12, 12, 11, 9$	$4 + 8 = 12$
偏移 2	$23, 21$	$(4 + 8) + (2 + 9) = 23$
偏移 1	44	$23 + 21 = 44$

同一轮交换读取的是其他通道为该次调用提供的寄存器值，随后各线程更新自己的局部变量。程序因此无需用共享内存模拟这些寄存器交换。交换调用本身不提供共享内存或全局内存访问的排序保证；跨线程束传递共享数组中的结果时，仍须使用相应的块级同步。

3.2 用线程束交换完成归约树的末段

一种渐进优化是保留共享内存树的前半部分，等到只剩较少的部分和时，再由第一个线程束完成归约。以下取 $B \geq 64$ 且 B 为二的幂，沿用上一节共享内存核函数的输入加载、共享数组 `values` 及其后的第一道屏障，把归约循环和输出部分替换为下面的代码。

代码 6 共享内存树结束后，把最后 64 个部分和交给一个线程束

```
for (unsigned int s = B / 2; s > 32; s /= 2) {
    if (t < s) values[t] += values[t + s];
    __syncthreads();
}

if (t < 32) {
    float value = values[t] + values[t + 32];
    value = warp_sum(value);
    if (t == 0) partial[blockIdx.x] = value;
}
```

循环条件 $s > 32$ 的含义是，最后一次共享内存层使用 $s = 64$，将 128 个部分和压缩为 64 个；对于 $B = 64$，初始加载后已经只有 64 个部分和，循环可以直接跳过。随后前 32 个线程先执行 `values[t] + values[t+32]`，完成必要的 $64 \to 32$ 合并，再用五轮交换完成 $32 \to 1$。若直接只读取前 32 项而省略这次加法，就会漏掉另外 32 个部分和。

当 $B = 256$ 时，初始加载先将 512 个输入变为 256 个共享值，随后共享内存层按 $s = 128, 64$ 运行两次；前 32 个线程接收最后 64 个部分和。块屏障数由直接共享树的 $1+\log_2 256 = 9$ 次降到 3 次：一次初始加载屏障，以及两次共享树层末屏障。

分支 `if (t < 32)` 只选择完整的第一个线程束，其他线程束不调用 `warp_sum`。这符合完整掩码的约定，因为掩码的范围是各自的线程束。需要严格保留的是所选线程束的全部 32 个参与线程，以及读取共享数组前已经完成的块屏障。当前片段还依赖 `values[t+32]` 有效，因此不能直接用于仅 32 个线程的块；这种配置可在初始加载后直接对各线程自己的值做一次线程束归约。

3.3 显式线程组与协作组

协作组（Cooperative Groups）接口把参与某次协作的线程集合表示为一个组对象。组可以对应整个网格、线程块簇、一个线程块，以及由线程块划分出的多线程束、单线程束或更小的线程组。这里使用由连续 32 个线程组成的分块组，使组内交换的参与范围直接体现在函数参数中。

使用协作组时，需要包含对应头文件，并由线程块成员共同取得块句柄和创建分组。下面的分组语句放在核函数中、任何只允许部分线程进入的分支之前；实际块大小取 32 的倍数。

代码 7 在核函数内由整个线程块共同建立 32 线程分组

```
// File scope:
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

// Inside the kernel, reached by every block thread:
cg::thread_block block = cg::this_thread_block();
cg::thread_block_tile<32> tile =
    cg::tiled_partition<32>(block);
```

这里每个线程得到的 `tile` 表示自己所属的 32 线程组。`tile.thread_rank()` 给出该线程在组内的编号，`tile.size()` 返回组大小，`tile.shfl_down(value, offset)` 则对应在组内向较大编号通道取值。原先的完整线程束辅助函数可以写成下面的形式。

代码 8 把参与交换的线程组显式传入归约函数

```
__device__ __forceinline__
float warp_sum_group(const cg::thread_block_tile<32>& tile,
                     float value) {
    for (int offset = tile.size() / 2;
         offset > 0; offset /= 2) {
        value += tile.shfl_down(value, offset);
    }
    return value;
}
```

分组操作 `tiled_partition<32>(block)` 是父组上的集体操作，必须由整个父线程块共同调用。因此，应先完成分组，再把 `tile` 传入归约函数。若把整块分组的创建语句放进一个只由第一个线程束调用的函数，就破坏了父组全员参与的条件。对于已经创建的 32 线程子组，后续归约只要求该子组的成员共同参与；其他子组可以执行自己的归约或别的工作。

协作组表达了通信范围，算法仍然需要正确地决定何时使用组内交换、何时进行跨线程束的共享内存同步。网格级或线程块簇级协作还有对应的执行条件；这里的普通块内实现只使用线程块与 32 线程子组。

3.4 先在各线程束内归约，再合并线程束结果

只优化最后一个线程束，仍会在共享内存树的前几层执行多次块屏障。两阶段线程束归约可以把更多的计算留在寄存器中：第一阶段，各线程束独立完成自己的局部归约；第二阶段，由第一个线程束归约各束的结果。跨线程束的数据交换只发生在这两个阶段之间。

令每块有 $Q = B/32$ 个线程束，束号为 $q = \lfloor t/32 \rfloor$，通道号为 $\ell = t \bmod 32$。首先，每个线程在寄存器中形成 $v_t = x_{s_b+t} + x_{s_b+t+B}$。线程束 q 对自己的 32 个 v_t 求和，由其第 0 通道把结果 u_q 写入共享数组。块屏障之后，第一个线程束的前 Q 个通道读入 $u_0, \ldots, u_{Q-1}$，其他通道取 0，再完成一次完整线程束归约。

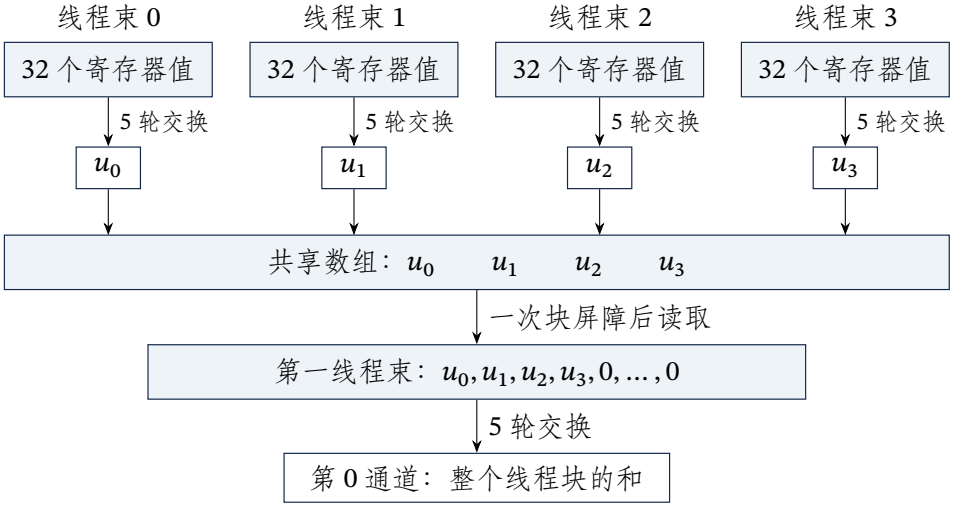


图 4 每块四个线程束的两阶段归约。各线程束先独立产生一个结果，只在共享数组交接处需要一次块屏障；第二阶段用零补齐到 32 个通道。

代码 9 两阶段线程束归约：一个块屏障与正确的第二阶段补零

```
template<int B>
__global__ void reduce_two_stage(const float* input,
                                 size_t n, float* partial) {
    static_assert(B >= 32 && B <= 1024 && B % 32 == 0,
                  "B must contain complete warps");
    constexpr int Q = B / 32;
    __shared__ float warp_totals[Q];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * B * blockIdx.x + t;

    float a = (i < n) ? input[i] : 0.0f;
    float b = (i + B < n) ? input[i + B] : 0.0f;
    float value = warp_sum(a + b);
    if (t % 32 == 0) warp_totals[t / 32] = value;
    __syncthreads();

    if (t < 32) {
        value = (t < Q) ? warp_totals[t] : 0.0f;
        value = warp_sum(value);
        if (t == 0) partial[blockIdx.x] = value;
    }
}
```

与前面的核函数一样，实际启动线程数必须等于 B，满足设备限制，且输出数组与输入不重叠。这里要求 B 是 32 的倍数，使第一阶段所有线程束完整；再要求 $B \le 1024$，使 $Q \le 32$，全部束结果可以交给一个线程束。由于第二阶段显式补零，Q 不必是二的幂，因此 $B = 96$ 这样的三个完整线程束也可以使用这套算法。

第二阶段的边界判断十分关键。例如 $B = 128$ 时，共享数组只有 $Q = 4$ 项；前 32 个线程如果无条件读取 `warp_totals[t]`，线程 4 至 31 就会越界。正确结构是先让四个通道读取真实部分和，其余通道设为 0，然后让完整的第一个线程束共同调用 `warp_sum`。只让前四个线程进入原有完整掩码函数，同样不能满足该函数约定。

唯一的块屏障保证所有 u_q 已写入共享数组，再由第一个线程束统一读取。最终写出只由线程 0 执行，各块写到自己的输出位置。对 $B = 128$，共享数组由直接共享树中的 128 个浮点数减到 4 个，即由 512 字节降到 16 字节；直接共享树的八次块屏障也减为一次。

这套代码中，各线程束都分别经历五轮交换，后面一些通道的运算结果不再进入第 0 通道的最终总和；第二阶段还包含对补零通道的运算。因而，按线程累计的标量加法次数会多于理想树的 $2B - 1$ 次必要合并。代码通过减少共享内存访问和块屏障来交换这些额外运算；其工作量仍随输入规模线性增长。这里应区分有效贡献通道减少与控制分歧：上述交换循环由全部通道无条件执行，通道利用率下降并不等同于源代码出现了新的按通道分支。

使用协作组时，核函数开始处先由全块建立子组句柄。每次束内归约均通过辅助函数

`warp_sum_group` 执行，将该句柄与当前部分和作为两个实参传入。两阶段之间保留块屏障，第二阶段由完整的第一个子组处理补齐后的束结果。

4 线程粗化与完整归约实现

4.1 增加每个线程承担的局部工作

前面的未粗化实现让每个线程先处理两个输入，再进入块内归约。对很大的输入，线程块数量可能远多于硬件能同时驻留的块数，块会分批执行。每批数据都会再次经历归约树从高并行度逐渐收缩到低并行度的过程，并支付相应的块内通信代价。

线程粗化（thread coarsening）让同一个线程先顺序合并更多输入，再执行一次块内归约。设粗化因子为正整数 C，则每个线程负责 $2C$ 个输入，每个块处理 $2CB$ 个输入，段首改为 $s_b = 2CBb$。每个线程在寄存器中形成

$$v_t = \sum_{j=0}^{2C-1} \tilde{x}_{s_b+t+jB}, \qquad \tilde{x}_i = \begin{cases} x_i, & i < N, \\ 0, & i \geq N. \end{cases} \tag{2}$$

这里 $\tilde{x}_i$ 表示逻辑补零后的输入，无需真的分配填充数组。完成局部累加后，只需再对 B 个 v_t 使用前面任一种块内归约结构。

地址 $s_b + t + jB$ 使同一轮 j 中相邻线程访问连续位置。以 $B = 128$、$C = 2$、$b = 0$ 为例，线程 5 读取下标 $5, 133, 261, 389$，线程 6 读取 $6, 134, 262, 390$。每个线程沿自己的序列顺序累加，而各线程的同一轮访问仍组成连续地址区间。图5 用较小的线程数展示这种结构。

块数随之变为 $G_C = \lceil N/(2CB) \rceil$。与未粗化相比，局部顺序工作增多，而需要启动和归约的线程块减少，块间原子更新数或写出的块结果数也随之减少。这个取舍在原本块数充足、硬件不得不分批处理时尤其有意义。

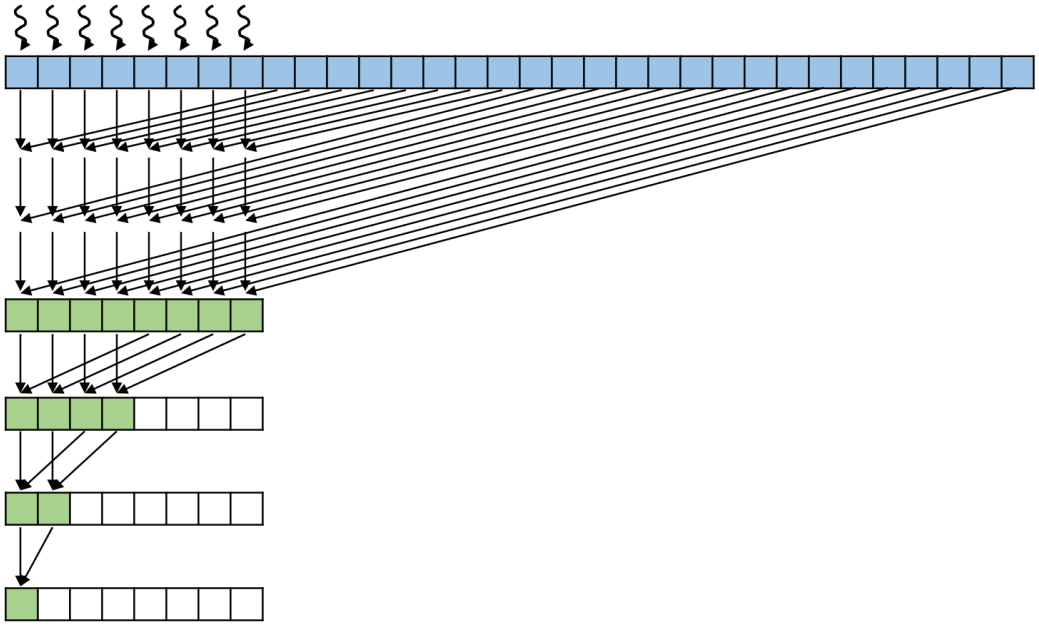


图 5 图示采用八个线程，每个线程先累加四个输入，再对八个局部部分和进行树形归约。上方的多轮读取保持跨线程的连续地址关系。

4.2 工作量、阶段数与同步次数的比较

为分析粗化如何改变树尾的重复开销，令原始每块输入数为 $M_0 = 2B$，并取 B 为二的幂。原始块内树深度为 $L_B = \log_2 M_0 = 1 + \log_2 B$。以下先使用单位加法时间、忽略数据传输细节的理想模型，比较处理同样 $2CB$ 个输入时的局部阶段数。

若未粗化的 C 个块必须由硬件依次执行，则需要约 CL_B 个阶段。粗化后，每个线程先把 $2C$ 个值顺序合为一个值，沿该线程的依赖链需要 $2C-1$ 次加法；之后再运行深度为 $\log_2 B$ 的块内树。因此，粗化后的跨度和必要工作量为

$$D_C = (2C-1) + \log_2 B = 2(C-1) + L_B, \tag{3}$$

$$W_C = B(2C-1) + (B-1) = 2CB - 1. \tag{4}$$

工作量仍恰好等于把 $2CB$ 个值合并成一个值所需的次数。未粗化的 C 个块内部一共做 $C(2B-1)$ 次必要加法，若再把这 C 个块和合成一个结果，还要做 $C-1$ 次，因此总数同样是 $2CB-1$。两种方案主要改变工作在线程内部、块内部和块之间的分配。

上述 CL_B 只计原有数据段依次完成块级输出的阶段，得到的是 C 个块和；粗化后的一个块直接输出覆盖同样数据的一个块和。实际总耗时还应考虑最后的块间合并。若原来的 C 个块本就能同时执行，则它们局部归约的并行深度只有 L_B，粗化反而增加了线程内部的顺序依赖，因此粗化因子增大不保证获得更短的时间。

以 $B = 128$、$C = 4$ 为例，原始每块有 $M_0 = 256$ 个输入，$L_B = 8$。四个块顺次执行的局部阶段数为 $4 \times 8 = 32$；粗化后，一个块处理 1024 个输入，每线程合并八个值需要七次加法，再经过七层块内树，共 $7 + 7 = 14$ 个阶段。这与式(3) 的 $2(4-1)+8 = 14$ 一致。$32/14 \approx 2.29$ 是这一理想模型中的阶段数比，并非实测加速比。

同步次数必须按实际块内实现分别计算。若采用每个树层末尾均保留块屏障的经典结构，C 个原始块共有 CL_B 次块屏障调用，粗化后只需对 B 个局部值执行一次归约，连同共享值初始化后的屏障，共 L_B 次。对于 $B = 128, C = 4$，计数从 32 次降至 8 次。若已经采用两阶段线程束归约，则每块原本只有一次块屏障，粗化把对应的 C 次降为一次，即这里的 $4 \to 1$。

因此，选择 C 时需要同时观察局部顺序依赖、块数和同步开销。较大的 C 能减少重复的块内归约及块结果写出，但也会减少可调度的块数；输入规模较小时，剩余块数可能不足以利用所有计算资源。循环展开及更多同时保留的中间值还可能提高寄存器占用。实际参数应依据固定输入规模下的运行时间比较，并保持相同的计时范围和数值正确性标准。

式(3) 使用的是理想二叉块内树。若采用固定 32 通道的两阶段交换代码，补零通道及不再进入最终结果的通道仍会执行部分指令，实际指令路径不必恰好等于这个理想层数。阶段数模型解释了粗化的来源，最终性能仍由所用代码与硬件共同决定。

4.3 完整核函数与块间合并

下面把每线程局部累加、两阶段线程束归约和最终输出连接起来。实现采用 C++17，使用前面定义的 `warp_sum` 辅助函数；源文件包含 `<cuda_runtime.h>` 和 `<cstddef>`。模板参数 B 表示每块线程数，C 表示粗化因子，编译期布尔参数 `AtomicOutput` 选择直接原子累加总和，或写出每个块的部分和。

对一个完整块，初始化直接读取每个线程的第一个输入，后面的循环进行 $2C - 1$ 次加法，与前面的局部跨度计数一致。尾块的读取分别检查边界，缺少输入的线程保留零或已有部分和，并继续参加完整的线程束归约。输入索引在乘法之前转换为 `size_t`，避免先用较窄整数计算大数组地址。

输出选择只在每块的第 0 个线程执行。采用原子输出时，输出只需一个浮点数，在所有块开始累加前初始化一次为 0；采用部分和输出时，输出需要 G_C 个浮点数，由各块分别覆盖自己的位置。两种模式均要求输出区间不与当前输入重叠。

代码 10 带线程粗化的完整归约核函数

```
template<int B, int C, bool AtomicOutput>
__global__ void reduce_coarsened(const float* input,
                                 size_t n, float* output) {
    static_assert(B >= 32 && B <= 1024 && B % 32 == 0,
                  "B must contain complete warps");
    static_assert(C >= 1, "C must be positive");
    constexpr int Q = B / 32;
    __shared__ float warp_totals[Q];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * C * B * blockIdx.x + t;

    float value = (i < n) ? input[i] : 0.0f;
    for (size_t tile = 1; tile < size_t(2) * C; ++tile) {
        size_t j = i + tile * B;
        if (j < n) value += input[j];
    }

    value = warp_sum(value);
    if (t % 32 == 0) warp_totals[t / 32] = value;
    __syncthreads();

    if (t < 32) {
        value = (t < Q) ? warp_totals[t] : 0.0f;
        value = warp_sum(value);
        if (t == 0) {
            if constexpr (AtomicOutput) {
                atomicAdd(output, value);
            } else {
                output[blockIdx.x] = value;
            }
        }
    }
}
```

原子模式可以使用下面的主机端启动函数。输入和输出指针均指向已经分配好的设备内存，块数应在设备允许的网格范围内。初始化与核函数在同一执行流（stream）中提交，使清零先于所有块的累加执行。

代码 11 在同一执行流中初始化输出并启动归约

```
template<int B, int C>
cudaError_t launch_atomic_sum(const float* input,
                              size_t n, float* sum,
                              cudaStream_t stream = nullptr) {
    cudaError_t status =
        cudaMemsetAsync(sum, 0, sizeof(float), stream);
    if (status != cudaSuccess || n == 0) return status;

    constexpr size_t chunk = size_t(2) * B * C;
    size_t blocks = n / chunk + (n % chunk != 0);
    reduce_coarsened<B, C, true>
        <<<static_cast<unsigned int>(blocks), B, 0, stream>>>
        (input, n, sum);
    return cudaGetLastError();
}
```

以 $B = 256, C = 4$ 为例，每块处理 2048 个输入。调用时将 `launch_atomic_sum` 的两个模板实参依次设为 256 和 4。空输入只执行输出清零，不启动零个线程块的核函数。启动函数是异步提交：返回的状态用于检查提交和启动错误；主机要读取数值时，还需等待该执行流完成并检查执行错误，再把设备结果复制到主机。

采用多轮部分和归约时，把 `AtomicOutput` 设为 `false`。空输入直接把最终结果设为单位元 0，单元素输入可以直接返回或复制该值，均无需进入多轮过程。以下递推取 $N \geq 2$，设当前输入长度为 n_k，每块容量为 $M = 2BC$，则下一轮的输入长度为

$$n_{k+1} = \left\lceil \frac{n_k}{M} \right\rceil, \qquad n_0 = N. \tag{5}$$

第一轮从原始输入写入临时数组，后续在两个互不重叠的临时数组之间交替读写，直到只剩一个部分和。每轮在同一执行流中依次提交，核函数边界为下一轮提供完整的块间阶段顺序。部分和输出模式会覆盖本轮的全部有效输出位置，因此这些输出槽位不需要像原子总和那样预先清零。

两种块间合并方法的主要差别是，原子模式用一次核函数完成所有块的局部计算，并竞争同一个最终地址；多轮模式把块间合并继续表示成较小的归约任务，需要额外的核函数启动和临时结果读写。输入规模、块数和设备特性共同决定其性能，比较时应包含所选方案完成一次归约实际需要的全部阶段。

5 综合算例与关键结论

5.1 非整块输入的完整归约过程

考虑 $N = 1000$ 个全部等于 1 的输入，取 $B = 128$、$C = 2$，使用两阶段线程束归约。每个块处理 $2CB = 512$ 个逻辑位置，所以需要 $\lceil 1000/512 \rceil = 2$ 个块。第 0 块处理下标 0 至 511，全部位置有效；第 1 块从下标 512 开始，只有 488 个真实输入，其余 24 个逻辑位置补零。

尾块线程 t 依次尝试读取 $512+t$、$640+t$、$768+t$ 和 $896+t$。前三次读取对所有 $0 \le t < 128$ 都有效；第四次读取要求 $896 + t < 1000$，即 $t \le 103$。因此线程 0 至 103 的寄存器部分和为 4，线程 104 至 127 的部分和为 3。边界处，线程 103 读取到最后一个有效下标 999，线程 104 的最后一次读取被边界判断跳过。

前三个线程束分别包含 32 个局部值 4，各得到 128。最后一个线程束包含线程 96 至 127，其中八个局部值为 4，二十四个局部值为 3，因此得到 $8 \times 4 + 24 \times 3 = 104$。与完整块相比，两块的中间结果如表5 所示。

表 5 1000 个全 1 输入在两个线程块中的归约结果

阶段或数量	第 0 块	第 1 块
有效输入数	512	488
逻辑补零数	0	24
四个线程束的和	128, 128, 128, 128	128, 128, 128, 104
第一个线程束第二次归约	512	488

在两个块的第二阶段，第一线程束均先读取四个共享部分和，再用 28 个零补齐为 32 项。它们共同产生块和 512 与 488。若使用原子输出，两个块分别向已经清零的总和地址执行一次原子加法，最终得到 1000。

若使用部分和输出，第一轮的结果数组是 $[512, 488]$，第二轮再对这个长度为 2 的数组归约，长度变化为 $1000 \to 2 \to 1$。第二轮仍可启动 128 个线程，只有线程 0、1 的首次输入读取有效，其余逻辑输入均取 0；所有线程继续遵守原有线程束与块屏障结构。第一次线程束归约得到一个 1000 和三个 0，第二次线程束归约得到最终的 1000。

这个算例说明，输入长度不足一个块或不足一个线程束时，仍可以保持固定的线程组织，只在数据加载处改变有效值与单位元的分配。有效输入完整参与、共享数组初始化以及完整线程束参与由此能够同时成立。

5.2 大规模输入需要多少轮核函数

考虑 $N = 1{,}000{,}000$，取 $B = 256$、$C = 4$，每块容量为 $M = 2048$。由于 $1{,}000{,}000 = 488 \times 2048 + 576$，第一轮有 488 个完整块和一个尾块，共产生 489 个部分和。尾块的段首为 999424，包含 576 个有效输入，其余 1472 个逻辑位置补零。

若继续用同样的配置归约部分和数组，第二轮只需一个块，因为 $489 < 2048$。完整的长度变化和块数如表6 所示。

表 6 百万元素的多轮分块归约

轮次	输入长度	每块容量	启动块数	最后一个块的有效输入数
第一轮	1,000,000	2048	489	576
第二轮	489	2048	1	489

若输入也全部为 1，第一轮前 488 个块各输出 2048，尾块输出 576；第二轮将这些值合为 $488 \times 2048 + 576 = 1{,}000{,}000$。这个输入规模在理想二叉树中具有约 20 层依赖，但多轮实现只启动两次核函数，因为每次核函数内部已经完成了每线程局部累加和块内的多级归约。核函数轮次、树的算术深度和块内屏障阶段数分别描述不同层次的工作。

若改用原子输出，同样的第一轮仍需 489 个块，但每块直接贡献一次原子加法，从而在一次核函数中完成全局结果。此时原子更新数量为 489，而逐元素原子方案需要 1,000,000 次。是否减少了一次核函数启动、是否保留最终同地址竞争，应一起计入方案比较。

5.3 归约结构与开销的联系

为统一比较，表7 取 $B \geq 64$ 且为二的幂，统计每块的同步阶段数；一次块屏障由参与线程共同执行，表中没有把它按线程重复累计。直接树保留前面代码中的末次屏障。两阶段方法本身只要求由完整线程束组成，表中的二的幂条件用于与其他树形实现采用相同配置。

表 7 不同块内实现的存储与同步开销

实现	中间存储	每块屏障阶段数	主要变化
递增步长全局树	全局数组	$1 + \log_2 B$	活动线程及地址逐层稀疏
递减步长全局树	全局数组	$1 + \log_2 B$	连续地址与连续活动前缀
共享内存树	B 个共享值	$1 + \log_2 B$	中间读写留在共享内存
共享树与末段交换	B 个共享值及寄存器	$\log_2 B - 5$	最后 64 项交给一个线程束
两阶段线程束归约	$B/32$ 个共享值及寄存器	1	各束独立归约，只在交接处同步
粗化与两阶段归约	$B/32$ 个共享值及寄存器	1	每线程先处理 $2C$ 个输入，减少块数

归约优化涉及三个相互联系的选择：怎样把二元合并分配给线程、在哪一级存储中传递部分和，以及让哪些线程共同同步。递减步长改善线程与地址的布局；共享内存和线程束交换逐步降低中间通信代价；线程粗化把更多工作放到单线程内部，以减少重复的块内归约。它们分别作用于不同的开销，因此可以组合使用。

正确性可以沿输入覆盖和阶段依赖追踪：每个有效输入进入一个且仅一个局部和，跨线程读取发生在所需结果已可见之后，每个块结果被全局归约恰好使用一次。改变存放位置、分组和同步范围时，这些关系需要同时保留。浮点输入还应按所要求的误差标准验证数值结果，因为不同的合法归约顺序可能产生不同的舍入误差。